

BitPattern: Enabling Efficient Bit-Serial Acceleration of Deep Neural Networks through Bit-Pattern Pruning

Gang Wang, Siqi Cai, Zhenyu Li, Wenjie Li, Dongxu Lyu, Yanan Sun, Jianfei Jiang, Guanghui He*

State Key Laboratory of Micro/Nano Engineering Science, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China

{wanganganJSTU, caisqi-2020, ambitious-lzy, wenjieli, lvdongxu, sunyanan, jiangjianfei, guanghui.he}@sjtu.edu.cn

Abstract—Bit-serial computation shows promise for accelerating deep neural networks (DNNs) by exploiting inherent bit sparsity. However, the original unstructured bit sparsity poses two major challenges for existing bit-serial accelerators (BSA): (1) workload imbalance from irregular bit distribution, and (2) inefficient memory access due to unpredictable non-zero bit locations. To address these issues, this paper proposes BitPattern, an algorithm/hardware co-design to efficiently accelerate bit-serial computation through bit-pattern pruning. At the algorithm level, we employ bit-pattern pruning to identify optimal combinations of predefined patterns and apply compression encoding to minimize weight storage. We further devise a pattern-similarity-based merging method to balance the bit-serial workload. At the hardware level, we co-design a bit-serial accelerator with a dedicated bit-pattern decoder and PE to leverage the potential of structured bit-pattern sparsity. The evaluation on several deep learning benchmarks shows that BitPattern can achieve $1.72\times$ memory reduction with negligible accuracy loss, and up to $2.11\times$ speedup and $1.86\times$ energy saving compared to state-of-the-art bit-serial accelerators.

Index Terms—Bit-serial computation, DNN acceleration, Sparsity, Algorithm/hardware co-design, Compression

I. INTRODUCTION

Deep neural networks (DNNs) have demonstrated remarkable success in various realms such as computer vision, natural language processing, and robotics. However, the superior performance of DNNs comes with rapid scaling of model size and computation complexity [1]. Such substantial burdens on memory and computation greatly hinder efficient DNN deployment in real-world scenarios, particularly on resource-constrained edge devices. Therefore, there is a strong need to prompt DNN efficiency from both software and hardware perspective.

Sparsification/pruning has been widely adopted in DNNs acceleration [2] to reduce the memory footprint and ineffective computations. Especially, structured pruning which targets spatially constrained non-zero value distribution of model weights can achieve a good tradeoff between model compression, workload balance, and hardware overhead. However,

This work was supported by the National Science and Technology Major Project of the Ministry of Science and Technology of China under Grant 2024ZD0802103 and the National Science Foundation of China under Grant 92464302. (*Corresponding author: Guanghui He).

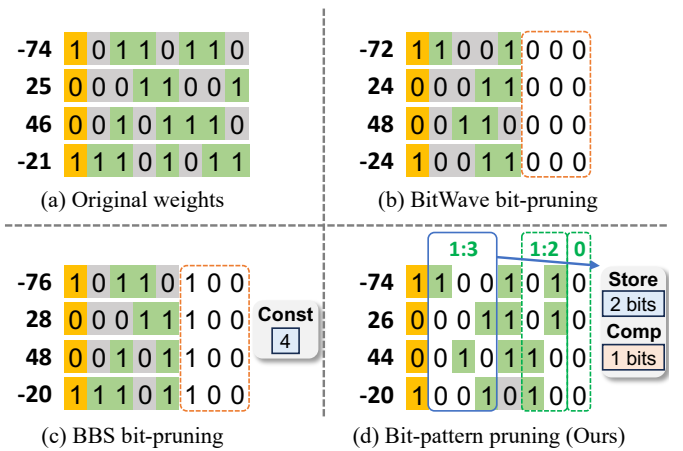


Fig. 1. Comparison of different bit-pruning methods for BSA. (a) Original weight values are represented in 2's complement. (b) Column-wise structured bit-pruning in BitWave [4] using sign-magnitude format. (c) Column-wise structured bit-pruning in BBS [5] using 2's complement format. (d) Bit-pattern pruning selectively prunes bits using predefined bit-patterns in sign-magnitude format. For example, a sparse bit-pattern (1:3) compresses the first 3 bits in current weight group into 2 storage bits per weight, requiring only 1 bit-serial computation cycle instead of 2, thereby reducing both memory consumption and computational workload.

such benefits require a time-consuming retraining process and access to the full training dataset or specialized training techniques [3]. Unfortunately, state-of-the-art (SOTA) DNN retraining is impractical due to its prohibitive computational costs and limited access to original training datasets, particularly given commercial restrictions and privacy regulations.

Bit-serial computation, which can leverage inherent bit sparsity in model weights by skipping zero bits, is a promising solution to relieve the exhaustive training cost and enhance computation efficiency [5]. Several bit-serial accelerators (BSA) [6] have been proposed to exploit bit sparsity serially. However, inherent unstructured bit sparsity presents two major challenges for BSA: (1) workload imbalance and (2) inefficient memory access. Firstly, the irregular distribution of non-zero bits (NZB) in original weights induce unexpected large number of NZB, leading to significant workload imbalance and hardware underutilization, since the BSA performance is

fundamentally decided by the highest number of NZB. While prior works such as BitPruner [7] and Bit-Balance [8] attempt to address this issue by enforcing maximum NZB constraints, they incur substantial training overhead. Furthermore, due to the unpredictable locations of NZB in full data width, these methods fail to optimize model parameter footprint and exhibit inefficient memory access, which remain critical bottlenecks in DNN acceleration.

Structured bit pruning has emerged as a promising approach to achieve both workload balance and memory access efficiency in BSA by structurally pruning the bits in model weights. Recent works such as BitWave [4] and BBS [5] implement column-wise bit pruning, which systematically eliminates entire bit columns by converting all NZB to zero, enabling the exclusion of these columns from both memory storage and computation, as illustrated in Fig.1(b)(c). These approaches demonstrate superior performance compared to conventional BSAs. However, BitWave’s fixed bit-position constraints in the column dimension suffer severe accuracy drop at high sparsity rates. BBS attempts to mitigate pruning errors through additional compensating constants which increases hardware complexity. Furthermore, neither approach optimizes the efficiency of remaining bit-serial computations, as they fail to skip zero-bit operations within compressed weights.

To tackle these problems, this paper proposes BitPattern, an algorithm/hardware co-design to efficiently accelerate bit-serial computation through bit-pattern pruning. The key insight of BitPattern is leveraging predefined bit-patterns with diverse sparse bit distribution to structurally compress weights and is capable of reducing bit-serial workload, offering better flexibility than column-wise bit pruning. For example, a sparse bit-pattern (1:3) compresses the first 3 bits per weight into 2 bits while requiring only 1 bit-serial computation cycle as shown in Fig.1(d). The main contribution of this paper are as follows:

- 1) We propose a bit-pattern pruning algorithm that identifies optimal pattern combinations through pattern fitting and error compensation, along with an efficient compression encoding scheme to minimize weight storage. We further introduce pattern-similarity-based merging to achieve reduced and balanced bit-serial workload.
- 2) We design a dedicated bit-serial accelerator with bit-pattern decoder and PE tailored with minimal hardware overhead via shared decoding patterns and direct shift mapping, harnessing the memory and computation advantages introduced by BitPattern.
- 3) Experiments on various DNN benchmarks are conducted. BitPattern achieves up to $2.11\times$ speedup and $1.86\times$ energy saving compared to SOTA bit-serial accelerators.

II. BACKGROUND

A. Bit-serial Computation

Recent work [9] demonstrates abundant bit-level sparsity in quantized models with integer-based representation, enabling computation skipping for zero bits. This zero-skipping

suggests a new computing paradigm, bit-serial computation, where non-zero bits are processed sequentially. A bit-serial PE computes the multiplication of an n -bit weight (W) with m -bit activation (X) as:

$$X \times W = \sum (-1)^{S_W} X(W_i < i), \quad (1)$$

where S_W is the sign of W . A set of BSA have been proposed to exploit bit sparsity. Pragmatic [6] utilize shift-and-add multiplications to skip zero bits while performing bit-serial computations. Bitlet [10] processes the serialized bits in a bit-interleaved manner and forward different bit-significances into separate shifter lanes. However, the performance of BSA is fundamentally determined by the highest number of NZB. The original weights typically exhibit irregular, unstructured bit sparsity with unexpected large number of NZB, leading to significant workload imbalance. To address this, Bit-Balance [8] use channel-wise quantization-aware training (QAT) to limit per-channel NZB. Nevertheless, this approach not only requires costly training process, but also retains weights at their original bit-width, failing to promote memory efficiency.

B. Bit Pruning

Model compression through pruning is a critical technique for reducing both memory footprint and computational complexity in DNNs. While conventional value pruning eliminates a portion of model parameters to enable compressed storage and skip redundant computations, its effectiveness in increasing zero-value density does not necessarily reduce the maximum number of NZB, which is crucial for BSA performance optimization.

In contrast, bit-level pruning offers a finer-grained approach that directly reduces bit-level computational workload of BSA. In particular, structured bit pruning systematically eliminates bit-serial operations while enabling efficient compression of remaining bits through structural patterns, thereby optimizing both computational throughput and memory utilization. Recent works have explored various approaches to structured bit pruning. BitWave [4] implements bit-column pruning across weight groups by iteratively eliminating NZB columns in sign-magnitude format, enabling efficient storage through non-zero column retention with minimal metadata overhead. Similarly, BBS [5] achieves compression by systematically pruning consecutive least significant bit-columns while maintaining accuracy through group-wise constant compensation. This paper introduces a novel structured bit-pruning methodology called bit-pattern pruning, which selectively eliminates bits based on predefined bit patterns for enhanced compression flexibility and efficiency.

III. PROPOSED BIT-PATTERN PRUNING

This section presents three key components in BitPattern algorithm: bit-pattern sparsity with compression encoding, output-oriented pruning algorithm, and pattern-similarity-based merging for workload balancing.

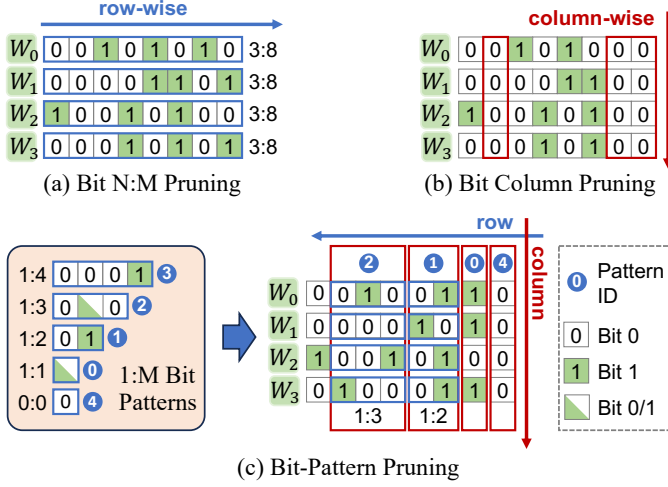


Fig. 2. Three structured bit pruning strategies are presented: (a) Bit N:M pruning balance the row-wise bit numbers within individual weights, e.g. 3 NZB in 8-bit weight. (b) Bit column pruning systematically eliminates NZB in a column-wise manner across multiple weights. (c) Bit-pattern pruning employs multiple 1:M bit sparse patterns, partitioning weights into column-wise segments and selectively pruning them using the most fitted row-wise patterns.

A. Bit-Pattern Sparsity and Compression Encoding

To generate more bit sparsity and achieve computational efficiency in BSA, three structured bit pruning strategies can be employed, as illustrated in Fig.2. Fig.2(a) demonstrates a structured bit-pruning approach which maintains a balanced row-wise distribution of NZB, as implemented in BitBalance [8]. This approach is referred to as bit N:M¹ pruning due to its fine-grained N:M sparsity characteristics. While bit N:M pruning effectively reduces NZB, it fails to compress memory footprint. Compressing N:M sparsity needs to store the non-zero elements and an additional mask to indicate the non-zero positions. This strategy becomes impractical when applied to bit-level N:M sparsity, as it would require per-data bit masking. Fig.2(b) depicts bit column pruning, which achieves model compression by eliminating entire bit columns across weight groups and storing only remaining columns with compact metadata for pruned positions. However, this rigid column-wise sparsity significantly degrades model accuracy at high sparsity rates and requires full bit-serial processing of remaining columns.

Fig.2(c) presents our bit-pattern pruning method, which leverages structured bit-pattern sparsity through predefined patterns. We utilize multiple 1:M bit patterns where each pattern specifies 0 or 1 NZB within an M-bit column-wise weight segment. For instance, as shown in Fig.2(c), the weight segment of bit[6:4] employs a sparse bit-pattern (1:3) with pattern ID ②. Our approach incorporates three pattern types: sparse bit-pattern for general compression, dense bit-pattern ① (1:1) to preserve critical bit distributions (e.g., bit[1] in Fig.2(c)), and zero bit-pattern ④ (0:0) for high compression in least significant columns (e.g., bit[0] in Fig.2(c)). Notably, the bit column pruning represents a special case of our method,

¹N non-zero bits in a consecutive block of M bits, where $N \leq M$.

TABLE I
THE COMPRESSION ENCODING VALUE TABLE OF BIT PATTERNS.

ID	Patterns	Original Values(W)	Compressed Values(W')	Bit Compression	Compressed Encoding
0	1:1	[0, 1]	[0, 1]	-	$0/\log_2 W + 1$
1	1:2	[1, 2]	[0, 1]	$2 \rightarrow 1$	$\log_2 W$
2	1:3	[0, 1, 2, 4]	[0, 1, 2, 3]	$3 \rightarrow 2$	$0/\log_2 W + 1$
3	1:4	[1, 2, 4, 8]	[0, 1, 2, 3]	$4 \rightarrow 2$	$\log_2 W$

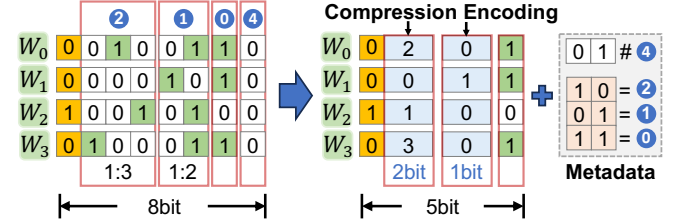


Fig. 3. An example for compression encoding of bit-pattern sparsity which prunes 3 bits per weight. A group of original weights are compressed into pattern specific encoding and metadata. The metadata contains (1) the number of least significant zero columns and (2) pattern IDs, where each pattern ID is encoded using 2 bits. The weights are stored in sign-magnitude format.

utilizing only dense and zero bit-patterns. This comprehensive design enables simultaneous optimization of row-wise workload balance and column-wise compression, effectively addressing the trade-off between computational efficiency and memory footprint.

Table I illustrates our compression encoding values of bit patterns. For instance, the sparse bit-pattern ③ (1:4) can efficiently compress the 4-bit original value (1,2,4,8) into a 2-bit encoding (0,1,2,3), achieving 2-bit compression. The encoding values are computed using $\log_2 W$, which not only provides optimal encoding efficiency but also enables direct compatibility with BSA shifter operations (e.g., $8 = 1 \ll \log_2 8$). Notably, all four bit-patterns in Table I maintain this $\log_2 W$ encoding property and require single bit-serial computation, facilitating hardware-efficient bit manipulation in BSA. Fig.3 demonstrates our compression encoding scheme with a bit-pattern combination including different bit-patterns. Given a combination, weight groups are compressed using pattern-specific encodings and metadata, which contains pattern IDs and zero-column number. Particularly, zero bit-pattern ④ are restricted to consecutive least significant columns, as eliminating higher significant columns exponentially impacts model accuracy. This approach enables efficient 2-bit pattern ID encoding for 4 bit-patterns, resulting in 8-bit metadata in this example. When more than three patterns are needed, we limit patterns to ① and ② to maintain consistent 8-bit metadata.

B. Output Oriented Bit-Pattern Pruning

To optimize the combinations of predefined bit-patterns which best fit the model weights, bringing larger bit-pattern sparsity and minimal pruning error, we propose an output-oriented bit-pattern pruning algorithm (Algorithm 1). Given a quantized weight matrix W and a per-weight pruning bit

Algorithm 1: Output Oriented Bit-Pattern Pruning.

Input : W : Quantized Weight, G : Group size,
 P : bit patterns, N : number of pruning bits
Output: $\hat{W}$: Compressed weights, M : metadata

```

1  $W_r = W.\text{reshape}(-1, G)$  ▷ step 1
2 Pattern Candidates:  $C \leftarrow \emptyset$ 
3 Generate all pattern combinations:  $PC = \text{comb}(P)$ 
4 for combination  $b \in PC$  do
5   if  $Pbit_b = N$  then
6      $C = C \cup b$ 
7  $Y = \text{zeros\_like}(W_r)$  ▷ step 2
8 for Group  $g \in [0 : G]$  do
9    $\text{bestError} = \infty$ 
10  for combination  $c \in C$  do
11    All possible value  $V$  under  $c$ 
12     $\text{Error} = |W[g] - V \otimes W[g].\text{sign}|^2$ 
13     $\text{minError} = \min(\text{Error})$ 
14    if  $\text{sum}(\text{minError}) < \text{bestError}$  then
15       $\text{bestError} = \text{sum}(\text{minError})$ 
16       $Y[g] = c$ 
17  $Y = \text{reshape}(Y, W.\text{shape})$  ▷ step 3
18  $\hat{W} = \text{zeros\_like}(W)$ 
19 for column  $j$  in  $W.\text{shape}[1]$  do
20    $\hat{W}_{:,j} = \text{Pattern-constraint value search}(W_{:,j}, Y_{:,j})$ 
21    $W_{:,j} = \text{Error compensate}(\hat{W}_{:,j})$ 
22  $M = \{ Y.\text{numZeroColumns}, Y.\text{IDs} \}$ 

```

number N , the algorithm first divides weights into groups and generates all feasible bit-pattern combinations with varying zero-column number, whose compressed bit-count equals N . In the second step, we determine optimal bit-pattern combinations for each weight group. A Kronecker Product $\otimes$ is used to generate all possible values associated with C . We compare generated values with the weight group to find the best matched values, and further sum up the errors to decide the group error for current combination c . This process is iterated through all combinations to search optimal pattern combinations Y that fit the pruned weights for each group.

However, step 2 optimizes pruned weights independently and may not guarantee minimal output error in $O = WX$. Therefore, we further propose a method to identify optimal bit-pattern combinations that can minimize $\|WX - \hat{W}X\|_2$ in step 3. Building upon the column-wise optimization framework introduced in GPTQ [11], our method iteratively determines optimal compressed weights and corresponding pattern combinations on a per-column basis, and compensates the pruning error to subsequent weight columns. Note that the weight column dimension (step 3) is orthogonal to grouping dimension (step 1). Incorporating group-wise pattern optimization within step 3 would severely degrade potential parallelism. To balance computational efficiency and pruning effectiveness, we constrain the search space of each weight value to the

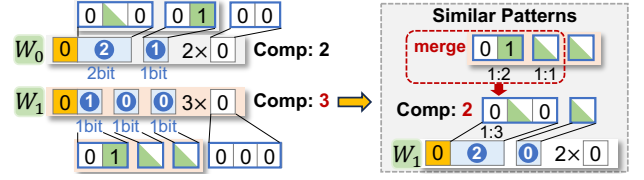


Fig. 4. Pattern-similarity-based merging example. Original W_1 compressed weight has three bit patterns which results in 3 bit-serial computation workload. Based on the pattern similarity, we can merge the first 2 patterns, 1:2 ① and 1:1 ②, into 1:3 ③, which only involve 2 bit-serial workload.

determined optimal patterns Y and perform the column wise value search similar to step 2 (line 20 in Algorithm 1). The pruned error are further compensated following [11]. Our pruning algorithm is free of training and can be directly integrated with quantization frameworks.

C. Pattern-Similarity-Based Merging

Despite employing bit-pattern pruning, bit-serial computations with compressed weights can still face workload imbalance during hardware acceleration. Fig.4 illustrates this challenge with two weight groups: W_1 requires two bit-serial computation cycles due to its two bit-patterns, while W_2 demands three cycles, resulting in workload imbalance. To resolve this inefficiency, we introduce pattern-similarity-based merging, which combines adjacent bit-patterns into a similar pattern while preserving the original bit-width. As shown in Fig.4, pattern ① (1:2) and pattern ② (1:1), initially compressing 3 bits and stored in 2 bits, are merged into a single pattern ③ (1:3), achieving two-cycle bit-serial computations. As such, the bit-serial workload is balanced and reduced from 3 to 2. While this merging process introduces pruning errors, they can be effectively compensated through step 3 of Algorithm 1. Furthermore, the pattern merging mechanism can also be efficiently implemented by refining similar pattern combinations in step 1 of Algorithm 1.

IV. HARDWARE ARCHITECTURE

In this section, we introduce the BitPattern accelerator architecture. Then, the bit-pattern decoder and corresponding bit-serial PE are proposed, which can efficiently exploit the bit sparsity after bit-pattern pruning.

A. Overall architecture

The overall BitPattern accelerator architecture is illustrated in Fig.5. The accelerator adopts an 32×32 bit-serial PE array with output-stationary dataflow, where each PE have 8 input lanes. The weights are broadcast vertically and the activations are broadcast horizontally. The input data are read from the DRAM interface to on-chip buffers and the output are stored in output buffers. BitPattern incorporates a metadata buffer to store the metadata after bit-pattern pruning. For each PE column, a BitPattern decoder is employed to decompress the compressed weights with metadata and generate corresponding bit-serial inputs.

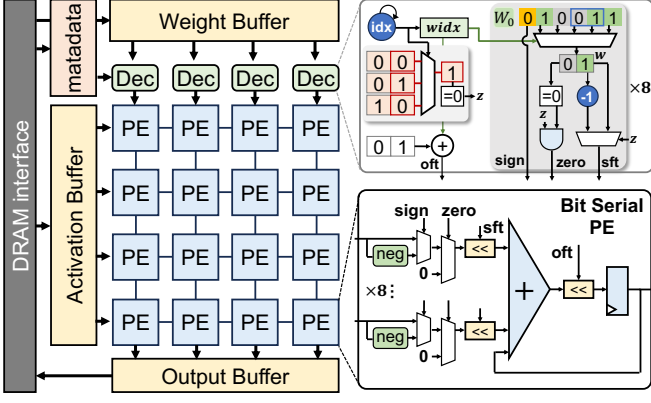


Fig. 5. BitPattern accelerator architecture.

B. BitPattern Decoder

The BitPattern decoder is shown in Fig.5, consisting of a metadata scheduler and 8 weight decoder for each PE input lane. An inner index register is used to schedule current pattern ID and weight bit-index $widx$. In each cycle, 1 bit-pattern is selected to decoding the weight segment w at current $widx$ into the shift values used in bit-serial PE. The compression encodings exhibit identical property between pattern ①/② and ①/③ according to Table I. Therefore, efficient encoding scheme discrimination is enabled through the least significant bit of pattern ID. Noticing that the compressed encoding values follow a systematic value of $\log_2 W$, $\log_2 W + 1$, or 0, where W represents the original weight segment, and the shift operations are inherently based on $\log_2(W)$. This structural relationship enables the shift values used in bit-serial PE to be directly extracted from compressed weights, requiring only a lightweight "-1" subtractor to compensate for the "+1" offset in compression encoding. Consequently, the selected pattern ID's least significant bit is compared with 0 to generate corresponding zero and shift values. Finally, the weight bit-index $widx$ is added with zero-column number to generate the common offset. Therefore, the BitPattern decoder exhibits decoding efficiency and minimal hardware through shared decoding patterns and direct mapping between compressed values and shift operations.

C. Bit-Serial PE

Bit-serial PE comprises of multiplexers, adders, shifters and accumulators to accomplish bit-serial computation. In each cycle, 8 activations are inputted and first perform the bitwise multiplication with signs and zeros signal from BitPattern decoder. The results are further shifted by the encoded value in a specific bit-pattern. After that, all shifted values are accumulated and shifted by the common offset. Since the bit range in each pattern is constrained, the first-stage shifters offer low hardware overhead. Fig.6 illustrates our bit-serial dataflow with bit-pattern decoder. In cycle 0, the first pattern ID ① is selected and the least significant of compressed weights are further decoded to the corresponding zero and shift values for bit-serial PE. While in cycle 1, the next pattern ② is used to decode the weight segment of bit[2:1].

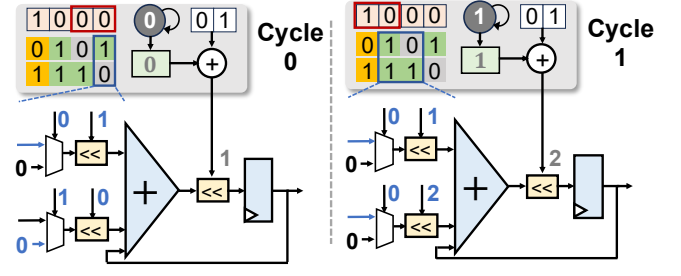


Fig. 6. An example of the bit-serial dataflow with 2 bit-patterns.

TABLE II
COMPARISON OF ACCURACY AND WEIGHT BIT-WIDTH BETWEEN BITPATTERN (BITP), BBS AND BITWAVE.

	ResNet-18	ResNet-34	ResNet-50	ViT-S	ViT-B
	Acc Bits	Acc Bits	Acc Bits	Acc Bits	Acc Bits
Float	69.76 -	73.29 -	76.14 -	81.39 -	84.53 -
BitWave	69.12 6.6	72.47 5.5	75.11 5.9	80.02 5.5	83.54 5.8
BBS	69.17 5.2	72.81 5.2	75.60 5.1	80.23 5.1	83.61 5.0
BitP	69.40 4.5	72.97 4.3	75.66 4.4	80.06 5.1	83.65 4.9
BitP-M	69.27 4.5	72.93 4.3	75.61 4.3	79.88 5.1	83.65 4.9

V. EVALUATION

A. Methodology

We evaluate BitPattern's effectiveness on various DNN models using ImageNet-1K [12], including ResNet [13] and Vision Transformer (ViT) [14]. Models are sourced from HuggingFace and PyTorch libraries and quantized to 8 bits. Our hardware evaluation compares the BitPattern accelerator against both the SOTA value-based accelerator ANT and bit-serial accelerators including Pragmatic, Bitlet, BitWave, and BBS (BitVert). The target number of pruning bits are set to 4. To achieve negligible accuracy loss, we retain 20% of the most sensitive channels and the remaining with 4 bit-columns pruned as detailed in [5] for BBS. For BitWave, 30% - 60% of the sensitive channels are retained, following [5]. For BitPattern, we apply a 2-bit pruning to the preceding layers to achieve a good tradeoff between model accuracy and inference efficiency under negligible accuracy loss.

All design results are implemented in SystemVerilog and synthesized using Design Compiler (DC) with TSMC 28nm technology at a clock frequency of 800 MHz. Power consumption is estimated using Synopsys PrimeTimePX. We implement a cycle-accurate simulator based on [15], with SRAM buffers modeled using CACTI [16] and DRAM memory access energy set to 1.2 pJ/bit, as reported in [17]. All acceleration arrays are configured with 1024 PEs and the same SRAM size.

B. Accuracy Comparison

Table II compares the accuracy and compressed weight bit-width of BitPattern against structured bit-pruning approaches in BBS and BitWave. The bit-width are calculated as the average bits per weight across all model layers, including associated metadata overhead. We denote **BitP-M** as BitPattern with pattern-similarity-based merging, which achieves more

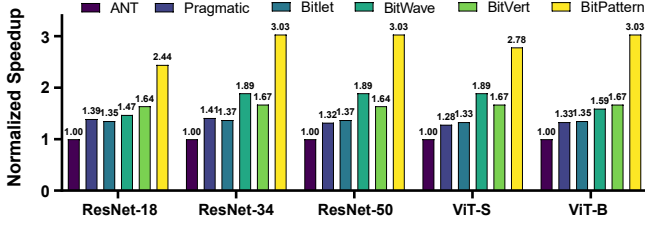


Fig. 7. The normalized speedup of different designs.

TABLE III
AREA AND POWER BREAKDOWN FOR BITPATTERN.

BitPattern	Configuration	Area(mm ²)	Power(mW)
PE	32×32	0.839	460.80
Decoder	32	0.007	5.26
Weight Buffer	128KB	0.426	104.96
Activation Buffer	256KB	0.832	145.41
Output Buffer	128KB	0.850	185.75
Metadata Buffer	8KB	0.027	6.56
Total	-	2.980	908.73

balanced bit-pattern distribution. BitWave requires a large portion of sensitive channels with higher bit-widths to maintain acceptable accuracy, primarily due to its highly dependency on natural zero-bit columns in original weights. While BBS achieves higher accuracy through constant compensation, it fails to exploit compression opportunities beyond least significant columns. Thanks to the flexibility of bit-pattern sparsity and proposed output oriented pruning, BitPattern achieves higher accuracy and lower weight bit-width across most cases. Specifically, it attains 1.72× and 1.73× model memory size reduction with an average accuracy loss of 0.67% and 0.75% under basic and pattern-similarity-based merging variants, respectively.

C. Hardware Comparison

We compare the speedup, energy consumption and efficiency with existing accelerators. The area and power breakdown of BitPattern are shown in Table III.

Fig. 7 presents normalized speedup comparisons between BitPattern and competing accelerators. While ANT reduces data bit-width through multi-type quantization, it fails to exploit bit sparsity. Both Pragmatic and Bitlet suffer from workload imbalance, limiting their acceleration efficiency. BitWave’s higher retention of sensitive channels increases both computation and memory access cycles. Although BitVert reduces sensitive channel count, it requires an additional cycle for sign bit processing in 2’s complement format. BitPattern achieves superior performance through pattern-similarity-based merging, requiring fewer and more balanced bit-serial workloads at equivalent compression width. Overall, our approach demonstrates significant speedups: average 2.86×, 2.13×, 2.11×, 1.66×, and 1.71× speedup over ANT, Pragmatic, Bitlet, BitWave and BitVert.

Fig.8 shows the energy consumption across different designs. BitPattern’s effective bit-pattern pruning reduces DRAM

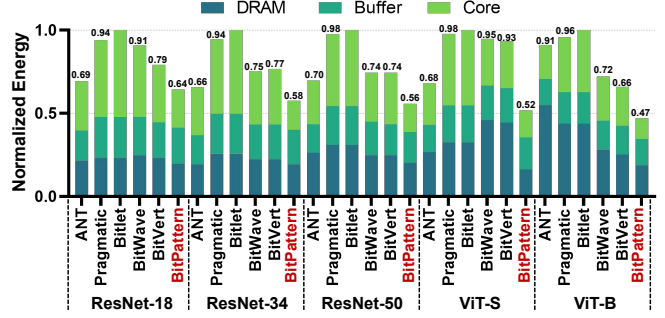


Fig. 8. The normalized energy breakdown of different designs.

TABLE IV
PE COMPARISON BETWEEN BITPATTERN AND PRIOR ACCELERATORS.

Design	PE Area (μm ²)	Power (mW)	Norm. Perf ¹	Norm. Perf/Area	Norm. Perf/Power
ANT	309.9	0.23	1.0×	1.00	1.00
Pragmatic	923.1	0.51	1.5×	0.50	0.68
Bitlet	1665.6	0.57	1.4×	0.26	0.56
BitWave	702.4	0.49	1.9×	0.84	0.89
BitVert	739.6	0.45	1.7×	0.71	0.87
BitPattern(Ours)	819.5	0.45	4.0×	1.51	2.04

¹ Normalized performance refers to the number of operations that can be executed on average per cycle in a layer.

and buffer access energy, achieving 1.25× lower access energy versus ANT. With pattern merging, our design demonstrates 1.82× and 1.73× core energy reductions compared to BitWave and BitVert. Overall, BitPattern achieves average energy savings of 1.31×, 1.77×, 1.86×, 1.49×, and 1.51× compared to ANT, Pragmatic, Bitlet, BitWave and BitVert, respectively.

Table IV compares the power and area efficiency between BitPattern and existing BSA at PE level. While all bit-serial PEs use 8 input lanes compared to ANT’s single 6-bit lane, BitWave and BitVert achieve efficiency by sharing one shifter across lanes, though BitVert requires additional BBS constant logic. Although BitPattern PE utilizes lightweight per-input shifters resulting in slightly larger area compared to BitWave and BitVert, it can efficiently skip zero-bit computations based on the predefined bit-patterns while BBS and BitWave can not skip zero-bit computations in the compressed weights. As such, BitPattern PE achieves 1.51×, 1.80×, and 2.13× improvements in area efficiency compared to ANT, BitWave, and BitVert.

VI. CONCLUSION

We present BitPattern, an algorithm/hardware co-design for efficient DNN bit-serial acceleration. Our approach combines structured bit-pattern pruning for optimal bit distribution with a specialized hardware architecture featuring dedicated bit-pattern decoders and processing elements. Evaluations demonstrate that BitPattern achieves up to 2.11× speedup and 1.86× energy reduction compared to SOTA bit-serial accelerators through simultaneous optimization of memory access and computation efficiency.

REFERENCES

- [1] A. Gholami *et al.*, “Ai and memory wall,” *IEEE Micro*, 2024.
- [2] Y. N. Wu *et al.*, “HighLight: Efficient and Flexible DNN Acceleration with Hierarchical Structured Sparsity,” in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2023, pp. 1106–1120.
- [3] C. Fang *et al.*, “An Algorithm-Hardware Co-Optimized Framework for Accelerating N:M Sparse Transformers,” *IEEE Trans. VLSI Syst.*, vol. 30, no. 11, pp. 1573–1586, 2022.
- [4] M. Shi *et al.*, “BitWave: Exploiting Column-Based Bit-Level Sparsity for Deep Learning Acceleration,” in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*. IEEE, 2024.
- [5] Y. Chen, J. Meng *et al.*, “BBS: Bi-directional Bit-level Sparsity for Deep Learning Acceleration,” *arXiv:2409.05227*, 2024.
- [6] J. Albericio *et al.*, “Bit-pragmatic deep neural network computing,” in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2017, pp. 382–394.
- [7] X. Zhao *et al.*, “BitPruner: Network pruning for bit-serial accelerators,” in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*. IEEE, 2020, pp. 1–6.
- [8] W. Sun *et al.*, “Bit-balance: Model-Hardware Co-design for Accelerating NNs by Exploiting Bit-level Sparsity,” *IEEE Trans. Comput.*, 2023.
- [9] G. Wang *et al.*, “BSViT: A Bit-Serial Vision Transformer Accelerator Exploiting Dynamic Patch and Weight Bit-Group Quantization,” *IEEE Trans. Circuits Syst. I*, vol. 71, no. 9, pp. 4064–4077, 2024.
- [10] H. Lu *et al.*, “Distilling Bit-level Sparsity Parallelism for General Purpose Deep Learning Acceleration,” in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 963–976.
- [11] E. Frantar *et al.*, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” *arXiv:2210.17323*, 2022.
- [12] J. Deng *et al.*, “ImageNet: A large-scale hierarchical image database,” in *IEEE Conf. Comput. Vis. Pattern Recognit.* IEEE, 2009, pp. 248–255.
- [13] K. He *et al.*, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [14] A. Dosovitskiy *et al.*, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *arXiv:2010.11929*, 2020.
- [15] C. Guo *et al.*, “ANT: Exploiting Adaptive Numerical Data Type for Low-bit Deep Neural Network Quantization,” in *Proc. 55th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. IEEE, 2022, pp. 1414–1433.
- [16] N. Muralimanohar *et al.*, “Cacti 6.0: A tool to model large caches,” *HP laboratories*, vol. 27, p. 28, 2009.
- [17] M. O'Connor *et al.*, “Fine-grained DRAM: Energy-efficient dram for extreme bandwidth systems,” in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2017, pp. 41–54.